Breaking the Kerckhoffs Geocache Cipher

A step-by-step walkthrough for the curious undergraduate

Abstract

Three geocaches hide a custom polyalphabetic cipher. The first publishes its whole table, the second hides the table but keeps two conveniences, and the third removes both conveniences and looks unbreakable. This document walks through how each part was solved, slowly enough to follow with basic linear algebra, a little statistics, and introductory algorithms. The point is not just the answer. It is to show how a handful of standard ideas—modular arithmetic, linear systems over finite fields, frequency statistics, bitmasks, and greedy algorithms—combine into a real cryptanalysis.

Contents

1 Roadmap: the whole solve on one page **2**

2 The setting: Kerckhoffs's principle **3**

3 Part 1: the table is public **3**
3.1 The machine . . . 3
3.2 Decryption by hand . . . 4

4 Interlude: searching a space too big to enumerate **4**
4.1 Optimisation problems . . . 4
4.2 Hill climbing . . . 4
4.3 Local optima and the two standard fixes . . . 5

5 Part 2: the table is hidden **5**
5.1 What is still visible . . . 5
5.2 Why the plaintext is "position plus rotation" . . . 6
5.3 An optimisation problem . . . 6
5.4 The score: a quadgram language model . . . 7
5.5 From hill climbing to simulated annealing . . . 7

6 Part 3: both conveniences removed **8**
6.1 Recovering the codeword dictionary . . . 8
6.2 The key and the model . . . 9
6.3 The linear structure that makes it solvable . . . 10
6.4 Solving the linear system . . . 10
6.5 Finding the crib: hints and a two-block search . . . 10
6.6 The plaintext and the final answer . . . 12

7 What was tried and why it failed **12**
7.1 Coordinate wording brute force 12
7.2 Frequency and difference statistics without a crib 13
7.3 Short cribs and underdetermined systems 13
7.4 Why the crib route works so much better 14

8 Putting it all together **14**

1 Roadmap: the whole solve on one page

The series is three increasingly hostile versions of the same machine. The table below is the shortest honest summary of the document; the rest of the text unpacks each row.

Part	What is public	What breaks it	Tools you will meet
1	the whole table; the first digit gives the length and the last gives the shift	nothing: decode directly	prefix-free codes, greedy parsing, modular arithmetic
2	the length digit and the shift digit; the table itself is secret	guess the table with a language model and local search	permutations, hill climbing, simulated annealing, n-gram language models
3	nothing but the ciphertext	recover the dictionary, then use a long crib and linear algebra	hash maps, expectation, linear systems over GF(2) and GF(13), bitmasks

Reading guide:

- Sections 2–6 cover the three parts in order. There is an interlude on hill climbing (Section 4) between parts 1 and 2, because part 2 is the first stage that is really an optimisation problem.
- Section 7 records the failed attacks. It is optional, but it explains why the obvious statistical ideas were not enough.
- Section 8 collects the concept-to-step map.

A pocket toolbox. Five ideas are used over and over.

1. **Modular arithmetic.** Letters are numbers in $\{0, \ldots, 25\}$ and every operation is modulo 26, so $25 + 1 = 0$.
2. **Permutations.** A table that assigns one codeword to each letter is a bijection, i.e. a permutation; there are 26! of them.
3. **Expectation.** The expected number of events is the sum of their probabilities over all opportunities. This is how we decide that a repeated 9-digit string is a real codeword rather than luck.
4. **Finite fields.** Modulo a prime p, every nonzero element has an inverse, so Gaussian elimination works exactly as over the reals. We use $p = 2$ and $p = 13$ because $26 = 2 \cdot 13$.
5. **Bitmasks.** A vector over GF(2) is a binary number and adding two vectors is XOR, which makes linear algebra on 52 unknowns fast enough to run hundreds of thousands of times.

2 The setting: Kerckhoffs's principle

Auguste Kerckhoffs argued that a cryptosystem should stay secure even if everything about its design is public except the key. The cache series is a playful demonstration. Each part uses the same basic machine:

1. There is a table mapping the 26 plaintext letters to 26 digit strings called *codewords*.
2. The table is used to replace each letter by its codeword.
3. After each codeword is used, the whole table is rotated by a shift value. The rotation is cumulative, so the same letter can be encoded by different codewords at different positions.

Part 1 publishes the table. Part 2 hides the table but keeps two hints in the ciphertext: the first digit of a codeword says how long it is, and the last digit says how much to rotate afterwards. Part 3 removes both hints: the first digit is meaningless and the shift is a secret value attached to each codeword.

Key idea

The whole series is a lesson in *known-plaintext attacks*. A cipher can look fearsome because the key space is huge, yet collapse the moment an attacker knows a long stretch of plaintext. That is exactly what happens in part 3.

3 Part 1: the table is public

Part 1 is the tutorial level: the cache page prints the whole table, so decryption needs no search at all. Seeing the machine work here makes the later versions much easier to read.

3.1 The machine

Let the public table be a list of 26 codewords, one per letter. In the actual puzzle the file alternates letters and codewords:

```
A 89411894
B 52381
C 88527391
...
Z 4502
```

Each codeword begins with its own length: `89411894` has length 8 and indeed starts with 8; `52381` has length 5 and starts with 5. This is a *prefix-free* code: no codeword is a prefix of another, because the first digit tells the decoder where the codeword ends.

After encoding a letter, the table is rotated so that every letter now maps to the codeword that used to belong to the previous letter. Concretely, if the table is a list of 26 codewords, a rotation by k moves each codeword down k slots (wrapping around). The rotation amount is the *last digit* of the codeword that was just used.

3.2 Decryption by hand

Because the first digit gives the length, decryption is a straightforward greedy scan:

1. Read the first digit; that is the codeword length.
2. Slice out the codeword.
3. Look up its plaintext letter in the current table.
4. Rotate the table by the codeword's last digit.
5. Repeat from the new position.

The decrypted message contains the final coordinates:

```
THE CACHE IS LOCATED AT NORTH THIRTY NINE DEGREES TEN POINT FOUR EIGHT TWO ...
```

What this teaches

Greedy decoding works because the code is prefix-free. The first digit is an explicit length field, so the parser never has to backtrack. This is the same idea as length-prefixed strings in binary formats.

4 Interlude: searching a space too big to enumerate

Part 2 is the first stage that cannot be solved by a direct scan. Before we meet it, here is the idea that rescues us, in the form you would meet in an optimisation or AI course — the material that usually comes after a first algorithms course.

4.1 Optimisation problems

An optimisation problem has three ingredients:

- a **state space**: the set of candidate solutions;
- an **objective function**: a number attached to every state, where larger is better;
- a **neighbourhood**: for each state, the states reachable by one small change.

The goal is to find a state with the largest objective value. If the state space is small we can enumerate it. If it has $26! \approx 4 \times 10^{26}$ elements, we cannot.

4.2 Hill climbing

Hill climbing (greedy local search) is the simplest idea that still works on huge spaces:

1. Start at any state, often a random one.
2. Look at its neighbours.
3. Move to the neighbour with the best score, if that score is better than where you are.

4. Repeat until no neighbour improves. Return the current state.

Think of a foggy hillside: you can feel the ground around your feet but you cannot see the whole landscape. Repeatedly stepping uphill reaches the top of *a* hill, not necessarily the highest one.

```
def hill_climb(state, score, neighbours):
    while True:
        best = max(neighbours(state), key=score)
        if score(best) <= score(state):
            return state
        state = best
```

4.3 Local optima and the two standard fixes

The landscape of a hard optimisation problem is usually full of *local optima*: states that beat all their neighbours but are not the global best. A plain hill-climber stops at the first one it meets. Two standard remedies:

- **Random restarts.** Run the climb from many random starts and keep the best result. Cheap, but each run still gets stuck.
- **Accept some bad moves.** Let the search step downhill occasionally so it can leave a bad basin and find a better one.

The second idea becomes *simulated annealing*, which part 2 uses: early on the search accepts many downhill moves and roams freely; as it cools, it accepts fewer and settles into a good basin.

What this teaches

Hill climbing is the bridge between "I can score a solution" and "I can find a good solution". State, score, and neighbourhood are problem-specific; the search loop itself is generic.

5 Part 2: the table is hidden

The interlude gives the shape of the attack — state, score, neighbourhood. Part 2 supplies the specifics. It keeps two of part 1's conveniences, which is exactly enough to make the problem hard but solvable.

5.1 What is still visible

Two things survive:

- The first digit of each codeword still says how long it is, so the ciphertext can be split into codewords by a simple scan.
- The last digit still says how far the table rotates after that codeword, so the cumulative rotation before every position is known.

5.2 Why the plaintext is "position plus rotation"

The equation that drives the whole series is worth deriving once, slowly.

Picture the table as a circular list of 26 codewords indexed by plaintext letter: index j is the codeword for letter j. Encoding letter j outputs the codeword currently sitting at index j; then the table rotates so that the codeword at index j moves to index $j + k$, where k is the last digit of the codeword that was just used.

Track one fixed codeword c. If it originally sat at index $\mathrm{pos}(c)$, then after a rotation by k it sits at index $\mathrm{pos}(c) + k$. Rotations compose, so after a total rotation S_i it sits at index

$$\mathrm{pos}(c) + S_i \pmod{26}.$$

Decoding is the inverse of encoding: the plaintext letter for c_i is simply the index where c_i currently sits, because the table is indexed by plaintext letter. Therefore

$$\boxed{p_i \;=\; \big(\mathrm{pos}(c_i) + S_i\big) \bmod 26} \tag{1}$$

where p_i is the i-th plaintext letter as a number $0, \ldots, 25$. The rotations are cumulative,

$$S_1 = 0, \qquad S_{i+1} = \big(S_i + \text{last digit of } c_i\big) \bmod 26,$$

and they are computable directly from the ciphertext.

A four-letter example. Suppose the alphabet is $\{0, 1, 2, 3\}$ and the table starts as $[c_0, c_1, c_2, c_3]$. Encoding letter 1 outputs c_1 and rotates by $k = 1$; c_1 now sits at index 2. If the next plaintext letter is 2, the encoder outputs whatever sits at index 2 now, which is again c_1. A decoder that meets c_1 with cumulative rotation $S = 1$ computes

$$\mathrm{pos}(c_1) + S = 1 + 1 = 2,$$

so it recovers plaintext letter 2. The equation is bookkeeping about where a codeword has travelled.

5.3 An optimisation problem

The shift sequence S_i is known from the ciphertext, so the only secret is the permutation pos. There are 26! possible tables—about 4×10^{26}—so brute force is hopeless. But the problem has exactly the shape from the interlude:

- **State:** a permutation of the 26 table entries.
- **Score:** how English-like the decrypted text is.
- **Move:** swap two entries of the permutation.

Score a state and you can hill-climb it. The next two subsections supply the score and then the search.

5.4 The score: a quadgram language model

The objective function from the interlude is a simple language model. A *quadgram* is a sequence of four letters, e.g. `THEM`. The repository includes counts of quadgrams from a large English corpus. If a candidate decryption contains quadgram q, we add $\log P(q)$ to its score. For a text of length n the total score is

$$\text{score}(p_1 \dots p_n) \;=\; \sum_{i=1}^{n-3} \log P(p_i p_{i+1} p_{i+2} p_{i+3}). \tag{2}$$

Logs are used for two reasons: they turn a product of probabilities (which would underflow) into a sum, and the sum is cheap to update when only a few letters change.

What this teaches

Language models are just probability distributions over short letter sequences. Turning a product into a sum of logs is a standard numerical trick, and it makes incremental rescoring possible.

5.5 From hill climbing to simulated annealing

Plain hill climbing would stop at the first locally best table. Simulated annealing is the "accept some bad moves" fix from the interlude. At each step we propose a swap; if the score improves we accept it, and if it gets worse we accept it with probability

$$\exp\left(\frac{\Delta}{T}\right), \tag{3}$$

where Δ is the score change and T is a *temperature* that starts high (explore) and slowly cools (exploit). This lets the search escape local optima early on and settle into a good solution later. The implementation runs several independent random seeds and keeps the best table, which is the random-restart remedy from the interlude.

The implementation is careful about speed. When we swap the entries for symbols a and b, only positions whose codeword is a or b change their plaintext letter, and only quadgrams overlapping those positions change. A precomputed list of positions per symbol (a hash map from symbol to positions) makes each move cost roughly the number of affected positions rather than the length of the text.

What this teaches

The algorithmic idea is *incremental evaluation*: never recompute more than the move actually changed. This is the same reason balanced trees and hash maps matter in algorithmic problem solving.

The annealing recovers the table and decrypts the message. It also reveals the coordinate instruction:

```
TO SOLVE SWAP AND DOUBLE THE LAST PARTS OF THE COORDS
```

Applying it to the posted coordinates $N\,39°\,10.227$, $W\,076°\,51.187$ means swapping the decimal minutes ($227/187 \to 187/227$) and doubling them ($187/227 \to 374/454$), giving the confirmed final

coordinate $N\,39^\circ\,10.374$, $W\,076^\circ\,51.454$. Applying the same recipe to part 1's numbers is a trap — the checker rejects it.

That is where the free information ends. Part 2 could still read the codeword length and the rotation off the ciphertext; part 3 removes both.

6 Part 3: both conveniences removed

Part 2 was hard but fair: the ciphertext still told us how to split it and how much the table rotated. Part 3 takes both away, leaving only digits.

Part 3 changes two rules:

- the first digit no longer gives the codeword length;
- the last digit no longer gives the shift: every codeword has a secret shift value in $0, \ldots, 25$.

Without the length prefix, the ciphertext cannot even be split into codewords without knowing the dictionary. Without the shift digits, the cumulative shift is part of the key. This section is the heart of the document.

6.1 Recovering the codeword dictionary

Part 3 hides the dictionary, so the first job is to find the 26 codewords. The text is still a concatenation of codewords; we simply do not know where the boundaries are. Two facts save us:

1. the code is still prefix-free;
2. each codeword repeats often. The table rotation spreads the plaintext letters over the codewords, so on average each codeword is used about once every 26 symbols — roughly 20 times in a 579-letter message.

Why prefix-free means uniquely decodable. Suppose no codeword is a prefix of another. Stand at the first digit and read forward until the digits you have seen form a codeword. You must stop there. If the true codeword were longer, it would have this codeword as a prefix, which the code forbids; and a shorter codeword would have been spotted earlier. So the first boundary is forced. The same argument applies from the next position, so the whole message has exactly one segmentation. This is why the length digit made parts 1 and 2 easy to parse, and why part 3 becomes parseable as soon as the dictionary is known.

A statistical argument for what is a codeword. To see why a repeated 9-digit string is not a coincidence, build a *null model*: pretend the ciphertext is a random string in which every digit is chosen independently and uniformly from $0, \ldots, 9$. Fix a pattern w of length L and ask how many times w should appear.

There are $N - L + 1$ places where w could start (a length-N string has that many windows of length L). In the null model the probability that the window starting at a fixed place equals w is $(1/10)^L$, because each of the L digits must match the corresponding digit of w. Expectation is linear — it does not require the windows to be independent — so the expected number of occurrences is

$$(N - L + 1) \cdot (1/10)^L \;\approx\; \frac{N}{10^L}.$$

For $N = 3377$ and $L = 9$ that is about 3.4×10^{-6}: a specific 9-digit pattern is essentially never expected to appear even once, let alone 18–26 times. The observed counts are overwhelming evidence that these strings are real codewords.

(There is a birthday-paradox subtlety here. With 10^L possible patterns and $N - L + 1$ windows, *some* pattern may repeat by chance at smaller L. The calculation above is for one *fixed* pattern, which is the right test for a candidate we have already singled out.)

Greedy segmentation. Once we have candidate codewords, "cut up" the ciphertext with them: start at the first digit, find a candidate that matches there, jump past it, and repeat. This is called *greedy* because each step commits to the current match and never backtracks. In a prefix-free dictionary there is at most one matching candidate at any position — if two different codewords both matched, the shorter would be a prefix of the longer — so the scan is forced, and the only question is whether it reaches the end. A candidate set that tiles the whole ciphertext with no leftovers is the dictionary.

The recovery procedure.

1. Count occurrences of every substring of length $2, \ldots, 9$ with a hash map. This is $O(9N)$ work.
2. Keep the longest frequent strings as candidates. A short string that lies inside a real longer codeword inherits that codeword's frequency, so preferring longer strings removes accidental prefixes.
3. Greedily segment the ciphertext with the candidates, as above.
4. Verify that the set tiles the entire ciphertext with no leftovers and that no candidate is a prefix of another. A failed check means a candidate was missing or wrong; adjust and repeat.

The result is 26 codewords, e.g.

```
243255912 222146337 946294413 80920926 63388773
37638005 6103003 0273351 2167948 8621836
...
860 781 418 0648 30 93
```

6.2 The key and the model

With the dictionary in hand we can split the ciphertext into codewords and reuse the position-plus-rotation model from part 2. The only change is that the rotation values are now secret too:

$$p_i = \big(\tau[c_i] + S_i\big) \bmod 26, \tag{4}$$

$$S_{i+1} = \big(S_i + \omega[c_i]\big) \bmod 26, \tag{5}$$

where

- c_i is the (now known) codeword index at position i;
- τ is a permutation of $0, \ldots, 25$ (the hidden table order);
- $\omega[c]$ is the secret rotation after codeword c.

There are 26 values of τ and 26 values of ω: 52 unknowns. The naive key space is $26! \cdot 26^{26}$, which is astronomically large.

6.3 The linear structure that makes it solvable

The secret shifts look intimidating, but the cipher is *linear*. The trick is to subtract the rule at consecutive positions. Both positions share the same cumulative rotation S_i, so the whole history before position i cancels and only the local rotation $\omega[c_i]$ survives:

$$\begin{aligned} p_{i+1} - p_i &= \big(\tau[c_{i+1}] + S_{i+1}\big) - \big(\tau[c_i] + S_i\big) \\ &= \tau[c_{i+1}] - \tau[c_i] + \omega[c_i] \pmod{26}. \end{aligned} \tag{6}$$

Everything on the right is a sum of key values with coefficients $0, \pm 1$. If we know a stretch of plaintext—a *crib*—then equation (6) gives one linear equation per adjacent pair of letters, plus one absolute equation for the first letter (that one does involve the cumulative shift, and it pins the history down). That is a linear system in the 52 unknowns over the ring $\mathbb{Z}/26\mathbb{Z}$.

Key idea

A crib of roughly 100 letters gives more than 52 equations. The system is overdetermined and usually has a unique solution for the key, even though the key space looked like $26! \cdot 26^{26}$. The apparent hardness was the missing plaintext, not the key size.

6.4 Solving the linear system

A correct crib therefore turns the cipher into a linear system. Solving it needs one algebraic observation. The modulus factors as $26 = 2 \cdot 13$, and both 2 and 13 are prime. By the Chinese remainder theorem, solving modulo 26 is the same as solving modulo 2 and modulo 13 separately and combining the answers. Over a prime field, ordinary Gaussian elimination works: we can divide by any nonzero coefficient.

1. Build the equations from the crib.
2. Perform Gaussian elimination modulo 2 and modulo 13.
3. If either system is inconsistent, the crib is wrong (or misplaced).
4. Otherwise read off free variables, enumerate their small number of possibilities, and keep only keys whose τ is a permutation.
5. Decrypt and score with the quadgram model.

What this teaches

Finite fields, Gaussian elimination, and the Chinese remainder theorem are not just abstract algebra. They are exactly the tools that turn a huge discrete search into a small linear-algebra computation.

6.5 Finding the crib: hints and a two-block search

Part 3 has no public table and no checker, so the crib is the whole game. The hint block eventually supplied:

[Puzzle 4] The two phrases `North Thirty Nine Degrees Ten Point` and `West Seventy Six Degrees Fifty One Point` both appear somewhere in the plaintext. No other assumptions about the plaintext need be made.

The earlier hint ("only the seconds differ") only guarantees that the final coordinate shares its degrees and minutes with the posted one. Hint 4 is what gives *exact* plaintext.

Write the two phrases as

$$A = \texttt{NORTHTHIRTYNINEDEGREESTENPOINT} \qquad (30 \text{ letters}),$$
$$B = \texttt{WESTSEVENTYSIXDEGREESFIFTYONEPOINT} \qquad (34 \text{ letters}).$$

Together A and B are 64 known letters, more than the 52 unknowns. The catch is that we do not know their offsets, and they are *not* adjacent and not in the same sentence position as in part 1.

Searching all offset pairs. There are about 550 offsets for A and 545 for B, so roughly 3×10^5 pairs. For each pair we build the combined linear system over $\mathbb{Z}/2\mathbb{Z}$ and test consistency.

Why check modulo 2 first? Because each equation can be packed into a Python integer: 52 bits for the coefficients and one bit for the right-hand side. A 53-bit integer fits in a machine word, and elimination on such rows is fast. The entire $\sim 3 \times 10^5$-pair search runs in about one second in ordinary Python.

```
MASK = (1 << 52) - 1 # 52 coefficient bits; bit 52 is the RHS

def consistent(a_pivots, b_rows):
    local = [0] * 53
    for row in b_rows:
        r = row
        while True:
            coeff = r & MASK
            if coeff == 0:
                if (r >> 52) & 1:
                    return False # reduced to 0 = 1
                break
            c = coeff.bit_length() - 1 # highest set coefficient
            p = a_pivots[c] or local[c]
            if p:
                r ^= p # GF(2) addition is XOR
            else:
                local[c] = r
                break
    return True
```

What this teaches

Choosing the right representation is an algorithmic superpower. "Vectors over GF(2)" become bitmasks, XOR becomes vector addition, and Gaussian elimination becomes a tight loop over machine words.

The search finds one pair that survives modulo 2 *and* modulo 13: A at symbol 46, B at symbol 114. The system still has 6 free dimensions modulo 2 and 6 modulo 13, so the key is not yet unique.

Reading the gap. Decrypting with any convenient representative of the solution space (free variables set to zero) exposes the text between the two phrases. It begins

`FIVEHUNDREDTHIRTEENINTHEOTHERDIRECTION`

That is the latitude seconds `FIVEHUNDREDTHIRTEEN` $= 513$ followed by the connector `INTHEOTHERDIRECTION`. Adding this 38-letter gap as a final crib pins the key uniquely. The recovered key is

$$\tau = [9, 16, 15, 11, 2, 12, 23, 19, 10, 18, 5, 1, 0, 17, 25, 3, 24, 7, 13, 4, 22, 20, 8, 6, 21, 14],$$
$$\omega = [17, 13, 5, 21, 21, 17, 2, 5, 17, 10, 4, 14, 0, 1, 11, 18, 14, 24, 1, 5, 1, 24, 11, 5, 18, 19].$$

6.6 The plaintext and the final answer

The decrypted message reads (spaced out for readability):

> WOW IM IMPRESSED. I THINK THIS PUZZLE IS PRETTY HARD. GO TO NORTH THIRTY NINE DEGREES TEN POINT FIVE HUNDRED THIRTEEN IN THE OTHER DIRECTION WEST SEVENTY SIX DEGREES FIFTY ONE POINT SIX EIGHT TWO. IM REALLY CURIOUS IF YOU CAN FIND A SOLUTION TO THIS GENERALIZATION THAT DOESNT RELY ON CRIBS. THE ONLY SOLUTION I FOUND USED THE WORD THE AS THE MOST COMMON TRIGRAM STARTING WITH EACH CODEWORD. EVEN THEN IT TOOK MILLIONS OF LETTERS, WASNT CONSISTENT, AND STILL NEEDED A LARGE EXHAUST. I REALLY THOUGHT I COULD USE THE FACT THAT THERE IS A STATISTICAL DIFFERENCE IN HOW FAR AWAY THE SECOND LETTER IS CIRCULARLY FROM THE FIRST IN ALL BIGRAMS REGARDLESS OF FIRST LETTER, BUT IT DIDNT WORK. CACHE IS MAYBE KNEE HIGH IN TREE.

The coordinate sentence gives

$$\boxed{N\,39^\circ\,10.513' \quad W\,076^\circ\,51.682'}$$

and the last sentence tells the finder where to look: knee high in a tree. The posted and final coordinates differ only in the seconds, exactly as the "only the seconds differ" hint promised.

7 What was tried and why it failed

The solve above looks inevitable in hindsight. It was not. It is worth recording the dead ends, because they show what makes the puzzle genuinely hard and how the successful method differs.

7.1 Coordinate wording brute force

The earliest attempts assumed the part 3 plaintext reused part 1's coordinate sentence verbatim, with only the numbers changed. We generated many wordings of "THE CACHE IS LOCATED AT NORTH ..." with plausible connectors ("YOULL ALSO NEED", "AND WEST", etc.), number

styles (digit-by-digit vs. number words), and coordinate candidates, and slid each one over every offset.

Why it failed. The hypothesis was too specific. Part 3 replaces the "THE CACHE IS LOCATED AT" prefix, uses "IN THE OTHER DIRECTION" instead of "YOULL ALSO NEED", and the two coordinate phrases are not adjacent. Every one of those differences makes the crib text wrong, and a wrong crib is rejected by the linear system. No amount of sliding fixes a wrong *word*.

7.2 Frequency and difference statistics without a crib

Several crib-free attacks were tried:

- **Letter-frequency annealing.** Optimise the key so that the decrypted letter frequencies match English.
- **Bigram-frequency annealing.** Same idea with two-letter frequencies.
- **Letter-difference distribution.** In any English bigram xy, the circular difference $y - x \bmod 26$ has its own distribution. Group the 676 codeword pairs by this difference and optimise the key to match the expected distribution.
- **Statistical recovery of the pair-difference table.** Because $p_{i+1} - p_i$ depends only on the codeword pair (c_i, c_{i+1}), the 26×26 table of pair differences is a Latin square. One can try to recover it from the observed pair counts using an assignment (Hungarian) algorithm. With only about 580 pairs and counts of 1–5 per cell, however, the counts are far too noisy to pin the table down.
- **Letter-difference trigrams.** Build the distribution of three consecutive differences from a quadgram corpus and anneal against it.

Why they failed. All of these objectives throw away the order of the text. They are low-dimensional summaries, so many wrong keys produce statistics that look plausible. The resulting optimisation landscape is full of local optima, and randomised search gets stuck. On synthetic data with a known key, none of these methods recovered the key. The author's own note in the plaintext says the same thing happened to him: even using `THE` as a crib took millions of letters, was inconsistent, and still needed a large exhaustive search. The statistical signal is real but far too weak on a 579-letter message.

7.3 Short cribs and underdetermined systems

Early attempts also tested short phrases from the hints (for example `ONLYTHESECONDSDIFFER`) at every offset. Each phrase is shorter than the 52 unknowns, so the linear system has more unknowns than equations: it is *underdetermined*, and almost every placement is consistent. The consistency test therefore filters nothing, and the solver can only report that the solution space is too large.

The lesson is a dimension count. For a unique key you need at least as many independent equations as unknowns, and in practice a comfortable margin. A 30-letter phrase gives 30 equations against 52 unknowns; two 30–40 letter phrases together give over 60 and become useful.

7.4 Why the crib route works so much better

The crib does not summarise the text; it gives actual letters. Each adjacent pair is a hard linear constraint. With enough constraints the key is forced, and linear algebra finds it exactly. Cribs turn a statistical search into a deterministic one.

What this teaches

When a system is linear, one long known plaintext beats any amount of statistical cleverness. Known-plaintext attacks are powerful precisely because they use exact information instead of aggregate information.

8 Putting it all together

The table below maps each stage of the solve to the standard topics it uses.

Step	Concept
Part 1 decryption	Prefix-free codes, greedy parsing, modular rotation
Optimisation background	State spaces, objective functions, neighbourhoods, hill climbing, local optima, random restarts
Part 2 table recovery	Permutations, simulated annealing, n-gram language models, incremental scoring
Dictionary recovery	Hash maps, substring counting, greedy segmentation, prefix trees, expectation and the birthday effect
Key model	Modular arithmetic, linear equations over a ring
Solving the key	Gaussian elimination, finite fields GF(2) and GF(13), Chinese remainder theorem, free variables
Offset search	Bitmasks, XOR as vector addition, bit parallelism, complexity analysis
Gap reveal	Partial decryption, solution-space representatives
Final answer	Coordinate arithmetic, the actual cache hunt

Key idea

The puzzle is a compressed tour of applied mathematics and computer science. It starts as a data-structures exercise (prefix codes and hash maps), becomes a statistics-and-optimisation exercise (language models and annealing), then turns into linear algebra over finite fields, and finishes with a bit-level search optimisation. None of the individual pieces is exotic. The difficulty is seeing how they fit together—and, in part 3, finding the one crib that lets the linear structure do its work.

Further reading

- Kerckhoffs's principle (Wikipedia) for the security philosophy.
- Practical Cryptography's quadgram page for the language model used here.

- Any introductory linear algebra text for Gaussian elimination; any abstract algebra text for finite fields and the Chinese remainder theorem.
- The repository files `src/part1.py`, `src/part2.py`, and `src/part3.py` for the actual code, and `POSTMORTEM.md` for the short technical summary.